

Sesión 11
Alta dimensionalidad y Selección de atributos

Docente: Soledad Espezua Llerena

s.espezual@up.edu.pe

Objetivos de la sesión

- Explicar por qué una alta dimensionalidad puede dificultar el análisis y el aprendizaje.
- Diferenciar **selección de atributos** y **transformación de atributos** como estrategias de reducción dimensional.
- Distinguir los enfoques **filter, wrapper y embedded**, reconociendo sus ventajas y limitaciones.
- Aplicar métodos básicos de selección de atributos e interpretar el subconjunto obtenido.

Docente: Soledad Espezúa Llerena

“More data beats clever algorithms, but better data beats more data”

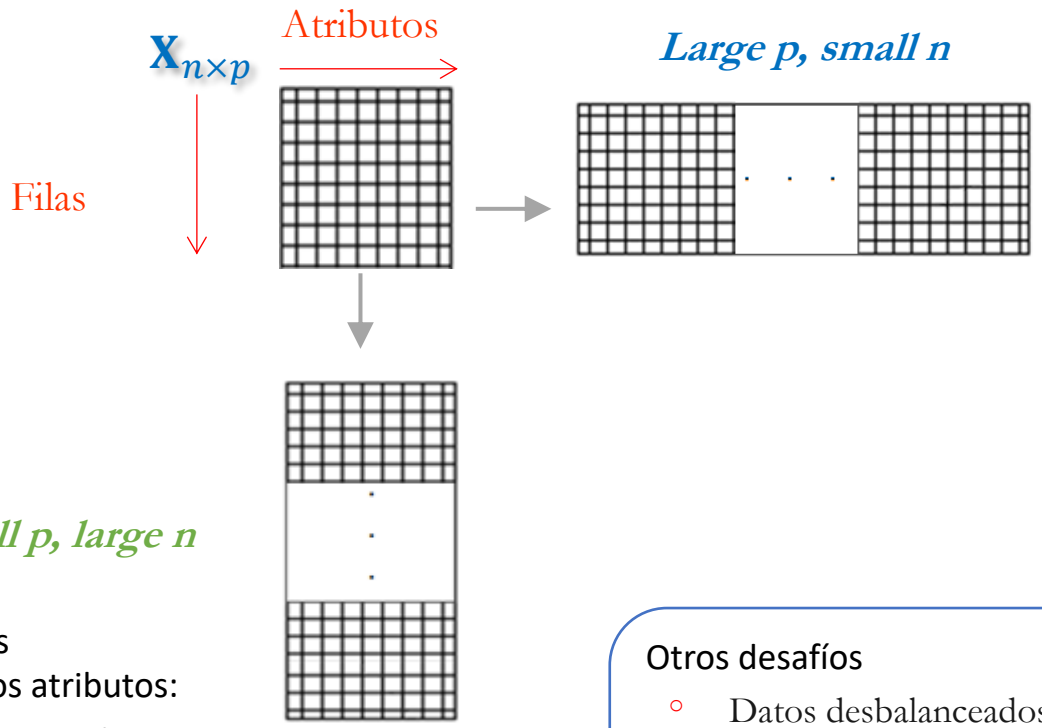
Peter Norvig

Priorizar la calidad de los datos

https://people.eecs.berkeley.edu/~russell/aima1e.html

“Inteligencia Artificial: Un Enfoque Moderno”
de Stuart J. Russell y Peter Norvig

Ingeniería de atributos



Alta dimensionalidad

Ejemplos con muchos atributos:

- Astronomía,
- Meteorología,
- Finanzas,
- Bioinformática.

Ejemplos con muchas observaciones y pocos atributos:

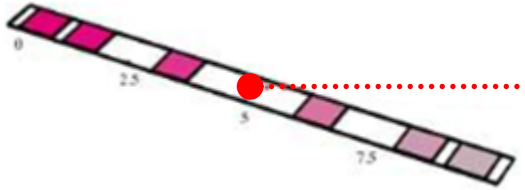
- Transacciones comerciales,
- sensores/IoT,
- telecomunicaciones,
- GPS,
- logs web.

Otros desafíos

- Datos desbalanceados (número de observaciones por clase)
- Valores faltantes
- Ruido / outliers

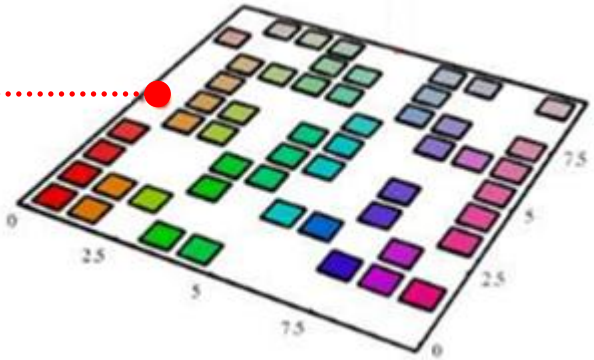


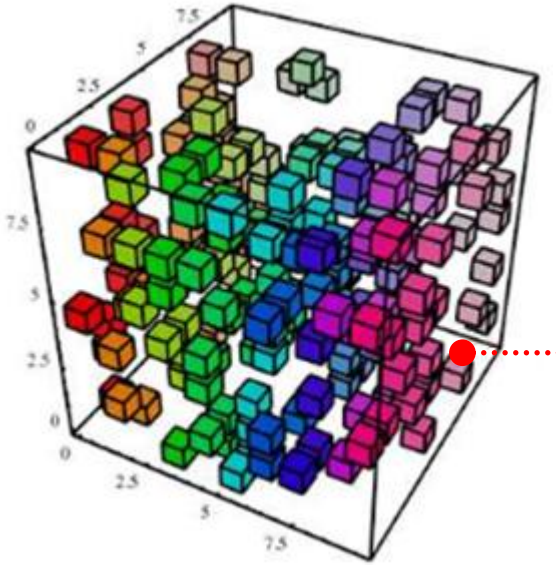
Alta Dimensionalidad



1 Dimensión:
10 posiciones

2 Dimensiones: 10x10
100 combinaciones





3 Dimensiones: 10x10x10
1000 combinaciones

Al aumentar el número de atributos, el número de regiones necesarias para cubrir el espacio crece exponencialmente.

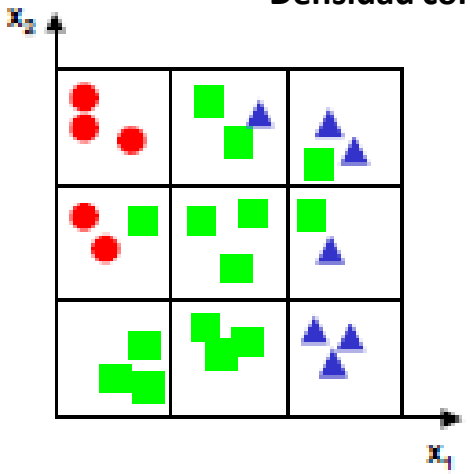
Imagen adaptada de Lopez Yse (2023)

Lopez Yse, D. (2023, June 30). *Straightforward guide to dimensionality reduction*. Pinecone

Alta Dimensionalidad

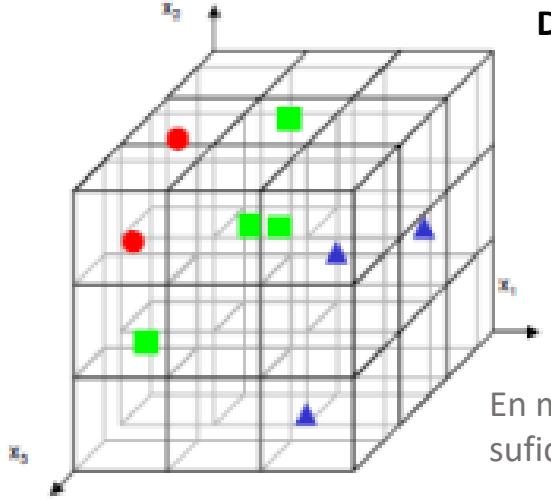
- Si el número de muestras **n** crece al mismo ritmo que el número de dimensiones **p**, es más probable que las celdas contengan datos y que la densidad se mantenga aproximadamente constante.

Densidad constante



- Si el número de muestras **n** se mantiene constante o crece lentamente mientras aumenta **p** el espacio queda cada vez más disperso y las distancias entre las muestras tienden a parecer similares.

Datos dispersos



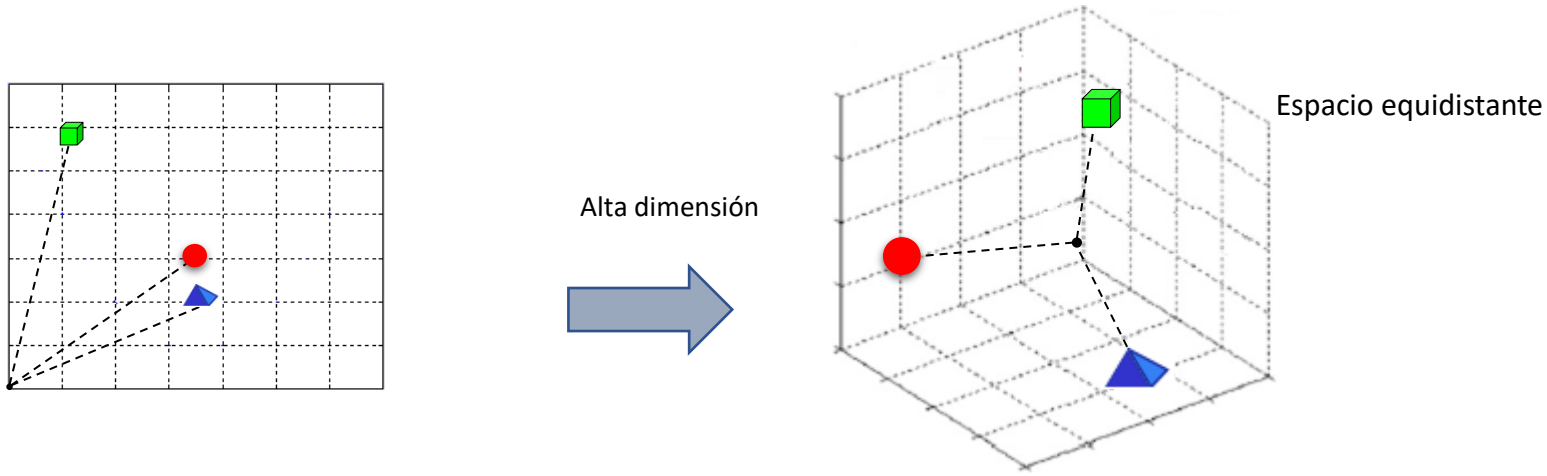
En muchas aplicaciones, conseguir suficientes observaciones no es viable.

.

Problema de "*large p, small n*": muchas variables y pocas observaciones

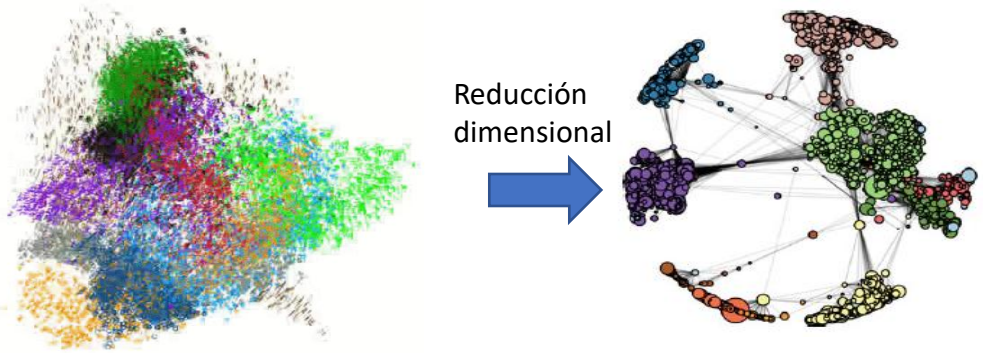
Maldición de la Dimensionalidad

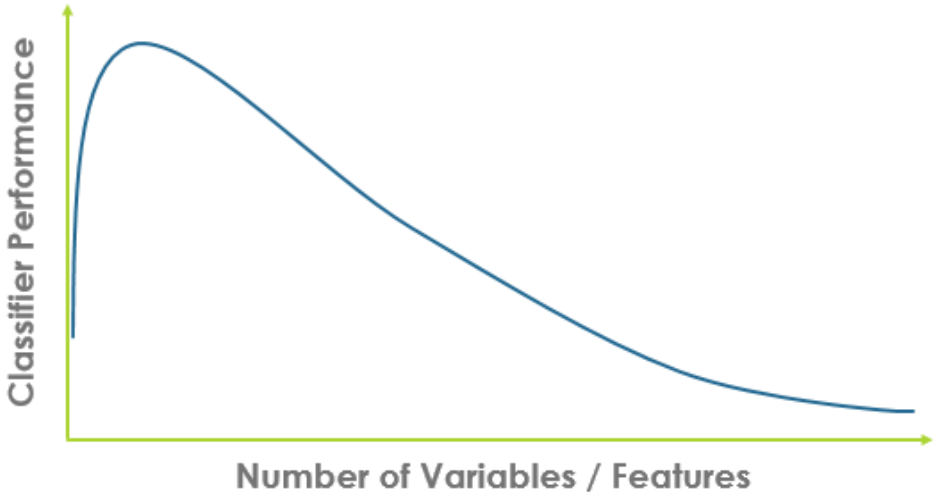
"A medida que aumenta la dimensionalidad del problema, especialmente cuando hay muchas variables en relación con pocas muestras, el espacio de representación se vuelve más disperso y las distancias entre puntos tienden a parecer similares"



- Espacio más **disperso**: con **n** fijo, las observaciones ocupan una fracción cada vez menor del espacio.
- Concentración de distancias: en alta dimensión, las distancias pueden volverse relativamente parecidas entre sí.
- En alta dimensionalidad, métodos basados en distancia como KNN, K-means o DBSCAN pueden tener dificultades para distinguir observaciones cercanas de lejanas, porque las distancias entre los puntos tienden a volverse relativamente similares.

Importancia



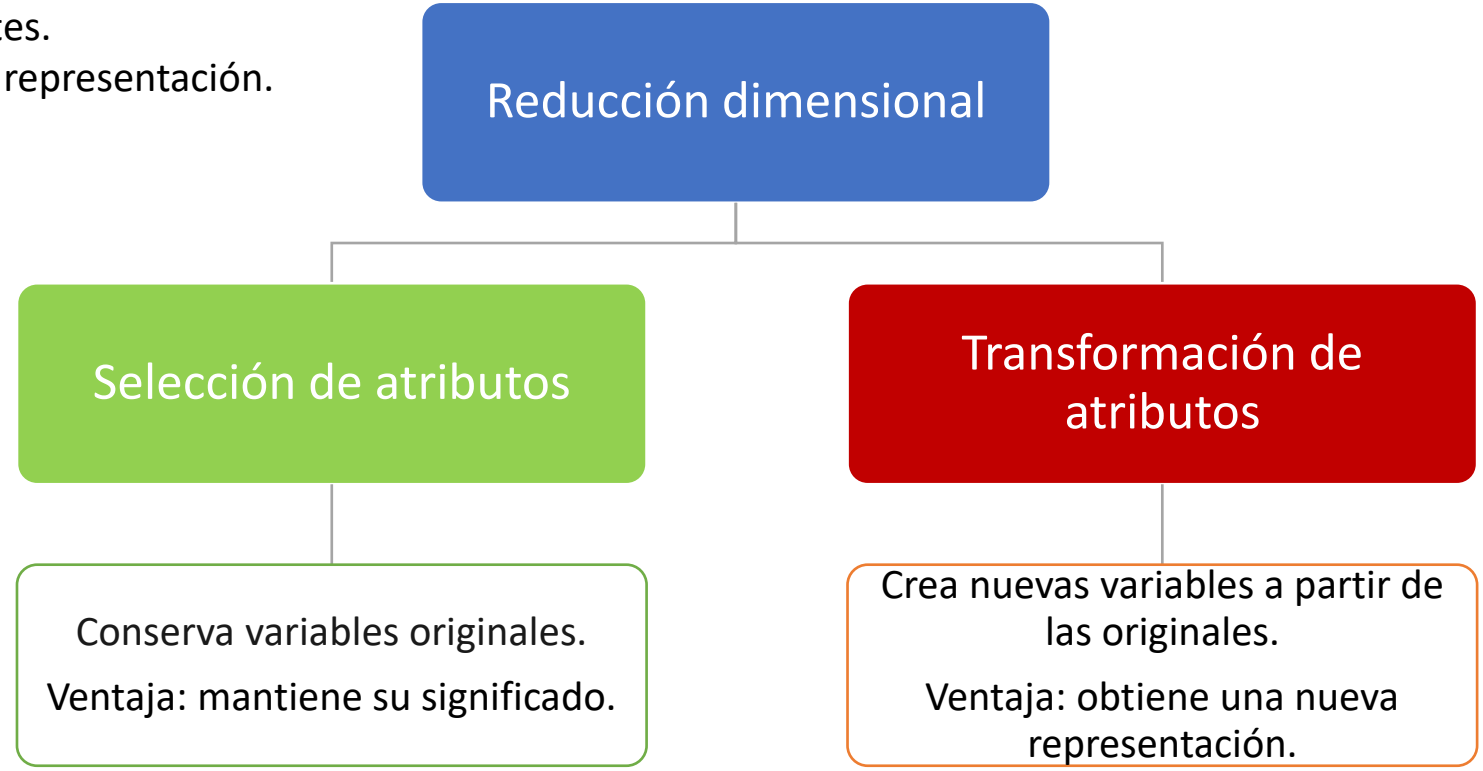


- Reduce redundancia e información irrelevante.
- Puede disminuir ruido y sobreajuste.
- Facilita visualización e interpretación.
- Reduce costo computacional.
- Puede mejorar la generalización del modelo.

Sin embargo, más dimensiones no siempre aportan más información útil; también pueden introducir redundancia, ruido y mayor complejidad.

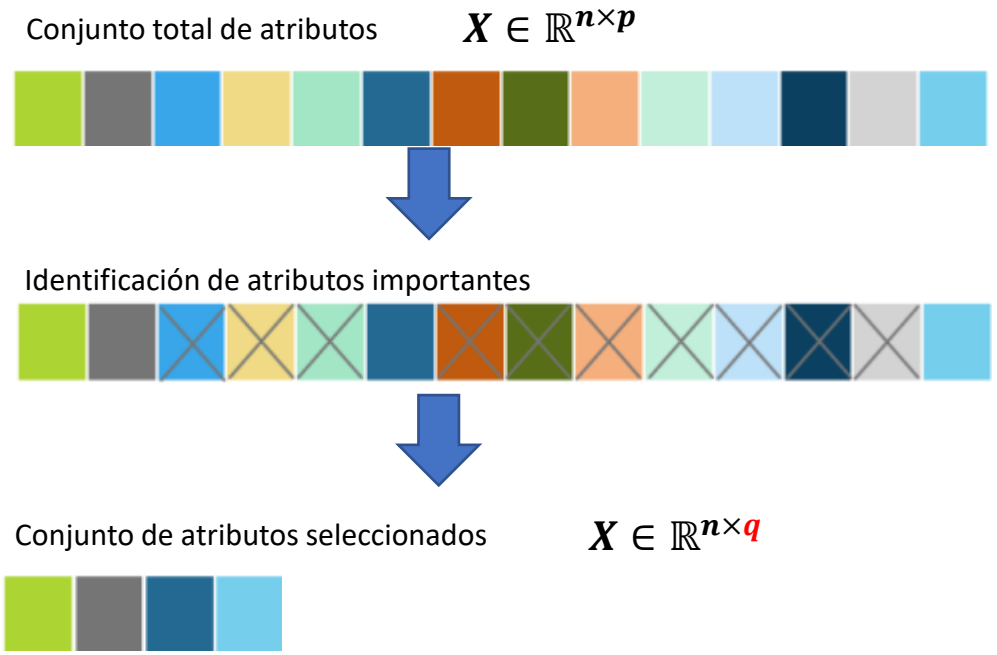
Reducción de la Dimensionalidad

Dos estrategias:
1. Seleccionar atributos existentes.
2. Transformarlos en una nueva representación.



Selección de atributos

- Proceso de elegir un subconjunto de atributos según un criterio de selección definido.



Ventaja

- Mantiene el significado de los atributos.

Estrategias de búsqueda de atributos

- Búsqueda exhaustiva
  - Evalúa **todos** los subconjuntos posibles. Puede encontrar el mejor según el criterio usado, pero su costo crece exponencialmente.
- Búsqueda secuencial
  - *Forward*: inicia sin atributos y agrega, en cada paso, el que más mejora el criterio.
  - *Backward*: inicia con todos y elimina, en cada paso, el que menos perjudica el criterio.
  - Son estrategias voraces: no garantizan el óptimo global.
- Búsqueda estocástica (Metaheurísticas)
  - Explora combinaciones mediante procedimientos aleatorios o heurísticos (p. ej., algoritmos genéticos). Puede encontrar buenas soluciones sin recorrer todo el espacio.

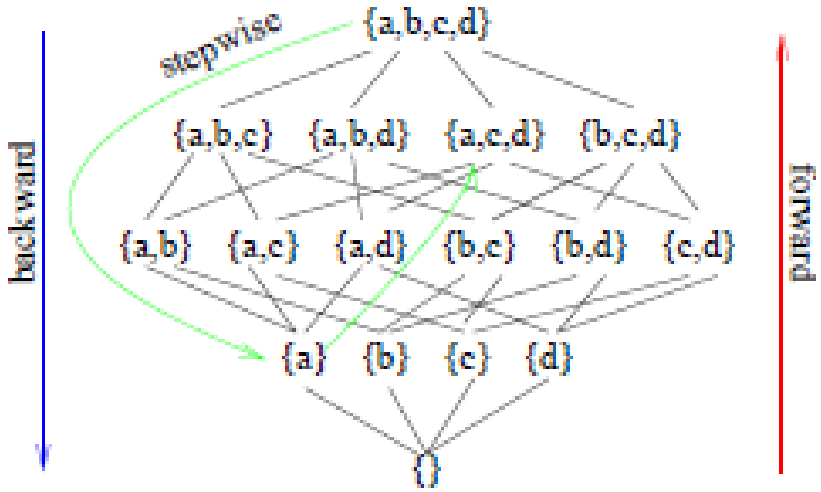
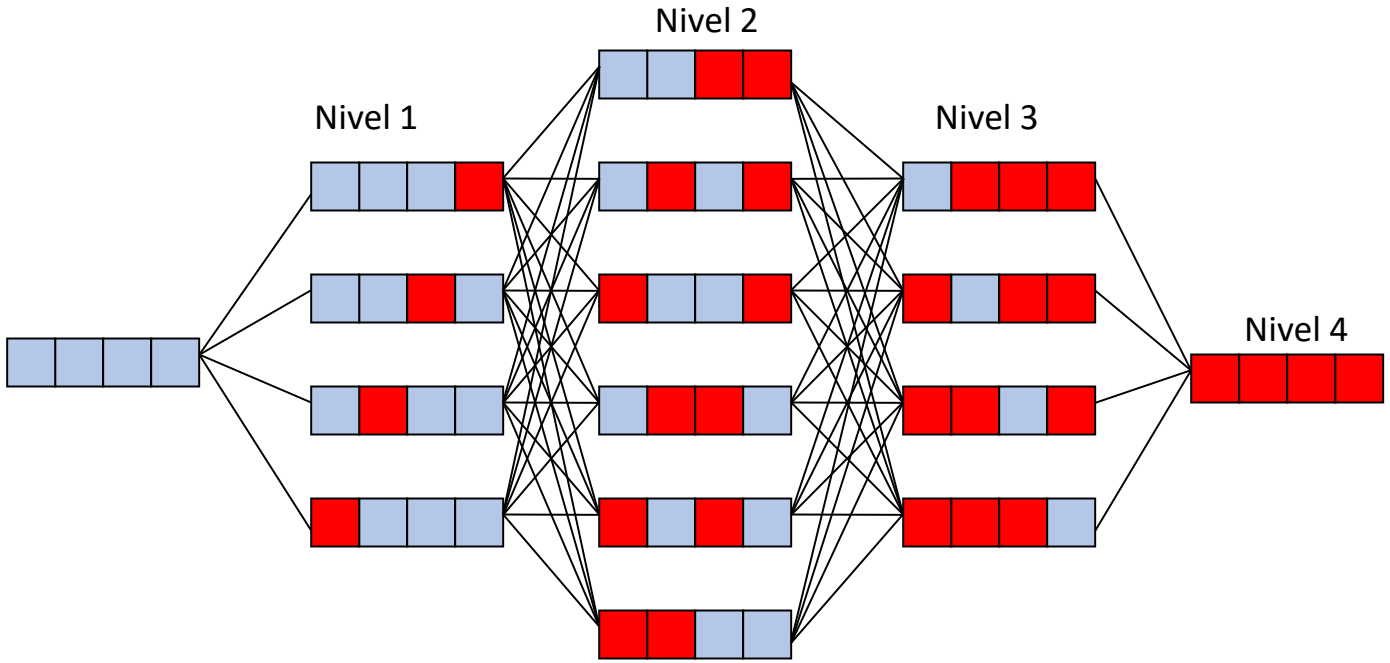


Figure 3.1: Backward selection, forward selection and stepwise selection.

Rüping, S. (2006). *Learning interpretable models* [Tesis doctoral, Universität Dortmund]. Eldorado. https://doi.org/10.17877/DE290R-8863

Selección de atributos

Ejemplo: Espacio de búsqueda exhaustiva de cuatro niveles



Nivel	Interpretación	k	Número de combinaciones
Inicio	Ningún atributo seleccionado	0	$\binom{4}{0} = 1$
Primer nivel	Se selecciona 1 atributo	1	$\binom{4}{1} = 4$
Segundo nivel	Se seleccionan 2 atributos	2	$\binom{4}{2} = 6$
Tercer nivel	Se seleccionan 3 atributos	3	$\binom{4}{3} = 4$
Final	Se seleccionan los 4 atributos	4	$\binom{4}{4} = 1$

- Búsqueda exhaustiva:

  $\binom{n}{k} = \frac{n!}{(n-k)!k!}$, n≥1 posibles espacios, donde:

  - **n:** número de atributos totales
  - **k:** nivel (número de atributos a combinar en cada subconjunto).

  Ejemplo: n = 100 y k = 10, $\approx 10^{13}$ subconjuntos

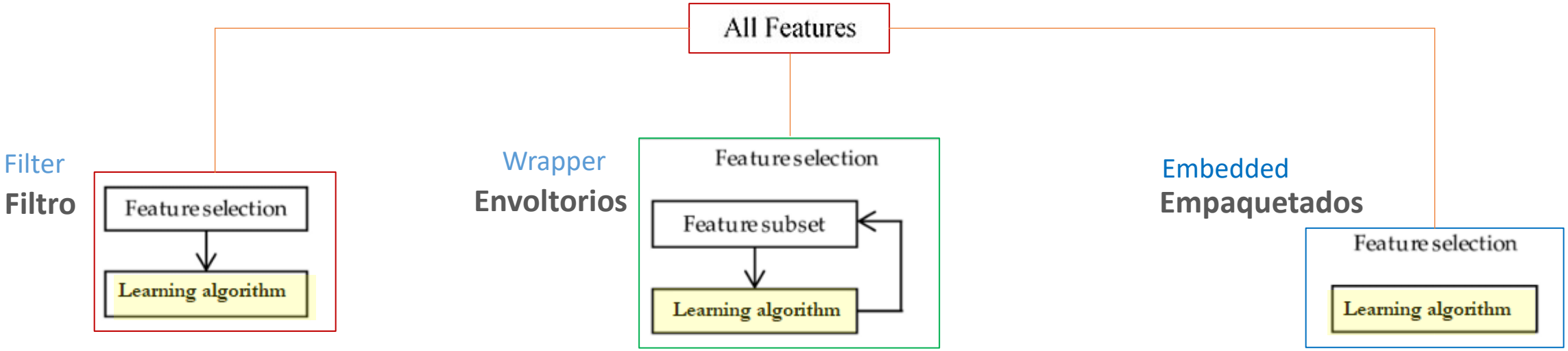
En total, el espacio de búsqueda tiene:

$$1 + 4 + 6 + 4 + 1 = 16$$

Es decir:

$2^4 = 16$ subconjuntos posibles de atributos

Métodos de Selección de atributos



¿Qué atributos parecen relevantes?

Asigna una puntuación a cada atributo usando criterios estadísticos o de información, sin depender del modelo predictivo final.

¿Con qué subconjunto funciona mejor el modelo?

Evalúa subconjuntos de atributos usando el desempeño de un modelo. Suele ser más costoso porque requiere múltiples entrenamientos.

¿Qué atributos considera importantes el propio modelo mientras aprende?

La selección del subconjunto de atributos está incluida dentro del entrenamiento del modelo. Ejemplos: regularización L1 y modelos de árboles con importancia de variables.

Filtro

Filter: puntúa atributos individualmente y luego selecciona los mejores.

En scikit-learn: SelectKBest + chi2, f_classif o mutual_info_classif.

- Chi^2 : El estadístico Chi-cuadrado evalúa la relación entre dos variables categóricas. Evalua la dependencia entre un atributo y la variable objetivo en problemas de **clasificación**.

$$\chi^2 = \sum \frac{(O_i - E_i)^2}{E_i}$$

Donde: Oi es la frecuencia observada y Ei es la frecuencia esperada.

- f_classif: Evalúa la importancia de un atributo comparando la varianza entre grupos con la varianza dentro de los grupos usando el valor F de ANOVA. Los valores más altos de F indican una mayor importancia del atributo.

$$F = \frac{\text{Varianza entre grupos}}{\text{Varianza dentro de los grupos}} = \frac{\sum_{i=1}^{k} n_i (\bar{X}_i - \bar{X})^2 / (k-1)}{\sum_{i=1}^{k} \sum_{j=1}^{n_i} (X_{ij} - \bar{X}_i)^2 / (N-k)}$$

Donde: *k* es el número de grupos; *ni* es el número de observaciones en el grupo *i*; $\bar{X}_i$ es la media del grupo *i*; $\bar{X}$ es la media global de todos los grupos; X_{ij} es la *j*-ésima observación en el grupo *i*; *N* es el número total de observaciones.

Método	Tipo de atributo	Tipo de variable objetivo	Qué evalúa
Chi-cuadrado	Categórico	Categórica	Asociación entre categorías
ANOVA	Numérico	Categórica	Diferencias de medias entre grupos

Filtro

Selección de atributos con un método Filter

1) Definir el criterio
`f_classif`: compara cada atributo numérico con las clases de `y`.

2) Puntuar los atributos
Cada atributo recibe un score **sin entrenar el modelo predictivo final**.

3) Seleccionar los mejores
`get_support()` indica qué atributos fueron retenidos.

```
from sklearn.feature_selection import SelectKBest, f_classif

# 1. Crear el selector: definir el criterio y cuántos atributos conservar
selector = SelectKBest(score_func=f_classif, k=5)

# 2. Evaluar cada atributo respecto de y
selector.fit(X, y)

# 3. Identificar los atributos seleccionados
seleccionados = X.columns[selector.get_support()]

print(seleccionados.tolist())
```

Seleccionar los 5 atributos numéricos más asociados con la clase

Filter evalúa los atributos antes e independientemente del modelo que utilizaremos posteriormente.

https://scikit-learn.org/stable/modules/classes.html#module-sklearn.feature_selection

Wrapper

- *Recursive Feature Elimination* (RFE)
  - Inicia con un conjunto de atributos, ajusta un modelo y elimina iterativamente los menos importantes.
  - Requiere un estimador que proporcione coeficientes o importancia de atributos (coef_ o feature_importances_).
- *Sequential Feature Selector* (SFS)
  - *Forward*: agrega atributos progresivamente.
  - *Backward*: elimina atributos progresivamente.
  - En cada paso evalúa el rendimiento del modelo, normalmente mediante validación cruzada.

Importante: RFE y SFS son procedimientos de búsqueda; no garantizan encontrar el subconjunto óptimo global.

Wrapper

Selección Wrapper con RFE

```
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression

# 1. Modelo que evaluará los atributos
modelo = LogisticRegression(max_iter=2000)

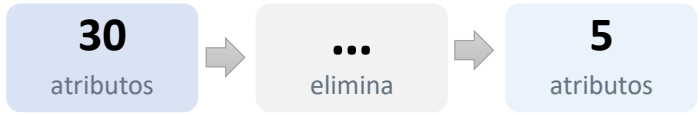
# 2. Crear RFE: escoger el estimador y pedir que conserve 5 atributos
selector = RFE(estimator=modelo, n_features_to_select=5)

# 3. Ejecutar la selección
selector.fit(X, y)
seleccionados = X.columns[selector.support_]

print(seleccionados)
```

Proceso iterativo

RFE usa un modelo repetidamente para ir eliminando atributos



1) Necesitamos un modelo
Lo usamos para evaluar la utilidad de los atributos.

2) RFE entrena y elimina
Entrena el modelo → identifica atributos menos importantes → elimina → vuelve a entrenar.

3) Resultado
Obtiene un subconjunto de atributos.

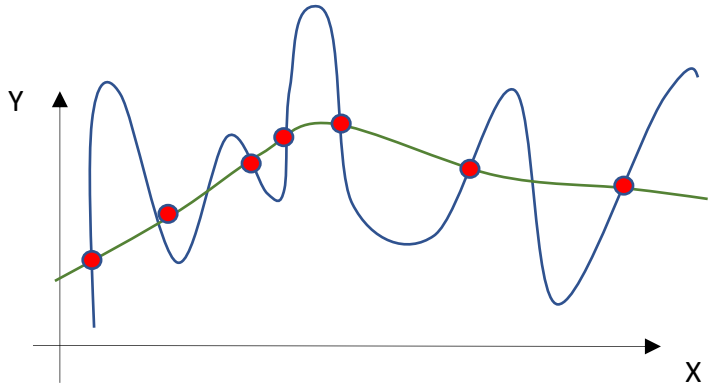
Wrapper selecciona según el desempeño de un modelo; por eso suele ser más costoso.

https://scikit-learn.org/stable/modules/classes.html#module-sklearn.feature_selection

Embeding

La selección está integrada en el entrenamiento del modelo.

- Regularización L1: Durante el ajuste, algunos coeficientes pueden quedar exactamente en cero. Esos atributos dejan de contribuir al modelo.

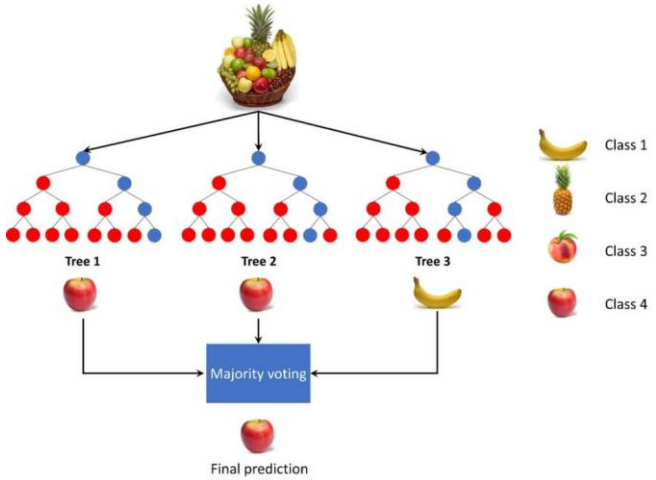


Regularización se usa para penalizar los coeficientes grandes.

Ventaja: selección y entrenamiento ocurren conjuntamente.

Limitación: el subconjunto depende del modelo elegido.

- Modelos basados en árboles: Árboles, Random Forest y Gradient Boosting producen medidas de importancia mientras construyen sus decisiones. Esa contribución puede usarse como criterio de selección.



Embeding

Selección Embedded con Random Forest

```
import pandas as pd
from sklearn.ensemble import RandomForestClassifier

# 1. Entrenar el modelo
modelo = RandomForestClassifier(n_estimators=100, random_state=1)
modelo.fit(X, y)

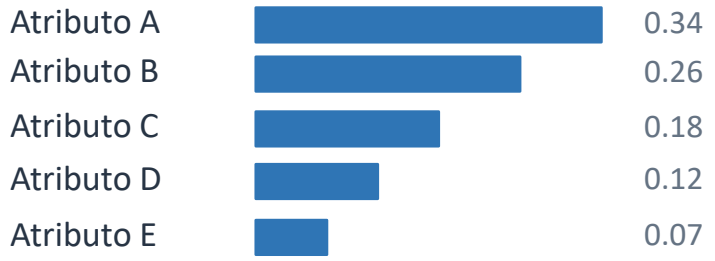
# 2. Recuperar la importancia aprendida
importancia = pd.Series(
                modelo.feature_importances_, index=X.columns
                ).sort_values(ascending=False)

importancia.head(5)
```

1) El modelo aprende a predecir
Utiliza los atributos disponibles para construir sus decisiones.

2) Durante el entrenamiento identifica contribuciones
`feature_importances_` indica cuánto contribuyó cada atributo al modelo.

3) Podemos seleccionar los más importantes
Por ejemplo, conservar los 5 de mayor importancia.



Mayor barra = mayor contribución al modelo

Random Forest no “borra” variables por sí solo: entrega importancias que podemos usar como criterio de selección.

https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomTreesEmbedding.html#sklearn-ensemble-randomtreesembedding

¿Qué cambió entre los tres enfoques?

	Filter	Wrapper	Embedded
¿Usa modelo?	No	Sí	Sí
¿Cuándo selecciona?	Antes del modelo	Probando subconjuntos	Durante el entrenamiento
Código clave	SelectKBest	RFE	feature_importances_
Costo	Bajo	Alto	Intermedio
Depende del modelo	No	Sí	Sí

Filter: ¿Qué atributos parecen relevantes?
Wrapper: ¿Con qué subconjunto funciona mejor el modelo?
Embedded: ¿Qué atributos considera importantes el propio modelo mientras aprende?

Estandarizar o escalar con min-max

Algoritmo / método	¿Admite negativos?	¿Escala importa?	Transformación conveniente
chi2	No	Sí	Min-Max
f_classif	Sí	Poco para ranking bajo escala lineal	Original / Standard / Min-Max
RFE + Regresión Logística	Sí	Sí	StandardScaler
SFS + Regresión Logística	Sí	Sí	StandardScaler
Random Forest	Sí	Poco	No imprescindible

UNIVERSIDAD DEL PACÍFICO

¡Gracias por su atención!